

『线性』复杂度的魔法——当PrefixSpan算法遇到了程序外部调用流程图

2026-08-12 目 约 4347 字 · 12 分钟

数学课上打盹的时候想出来的，一种在反病毒领域可能会有奇效的PrefixSpan有向图变种算法，可以达到线性级别的复杂度，目前应用在CSafe-Starlight V3系列引擎的tosSPM语义解析引擎上。引擎的分析能力似乎还不错，也算是有数据证明了这个猜想是成立的吧。

事情起源于CSafe-Starlight V3系列反病毒引擎的研发工作。

v3系列引擎的架构，我大概从上学期期末之前就开始设计了。最初的想法是提取程序的外部调用森林，即对程序进行线性反汇编和简单的跳转模拟，对程序的每个入口点向下延伸出的所有可达的导出函数调用进行分析，然后解析出一个拥有多个入口点，也就是多个根节点的外部调用森林，在这个森林上执行带跳过（是为了规避混淆）的频繁调用链挖掘（由于目标数据结构是森林，因此设计一个带混淆跳过的树上运行的频繁序列挖掘算法并非难事，我当时自己写了一版设计，后来发现和PrefixSpan是同一个原理），然后对挖掘出的频繁恶意/良性调用链建立数据库，推理的时候再进行匹配与加权。

这个时候，想必接触过程序外部调用流图，也就是EFG（External Flow Graph）的同学应该看出来了，这套想法是相当天真稚嫩且不可靠的。核心原因在于一点——对于一个反汇编后的PE程序，其外部调用之间的关系是非常复杂的，要将一个程序的所有可能的跳转树全部展开，在外部调用森林生成部分，程序就会卡死（结合后文就很容易发现，这本质上是一个将有向图带去重地强制展开为树状结构的想法，而解决这个问题的算法时间复杂度是指数级别的）。

其实初版设计的结局也是不难想到的~毕竟，如果真的可以让我就这么轻易地在有限的时间复杂度内，从那么庞大的程序样本库中提取出准确的频繁调用链来（至少在我看来，调用链算是一个特别有用的静态分析特征了），那肯定早就有相关的研究和实践了。

怎么办呢？ternaryop8479也想不到其他办法，那就只能抛弃这个思路，硬着头皮使用EFG了。

不过~在有向图（EFG外部调用流图本质上其实就是一个有向图）上执行（带去重的）频繁链条挖掘属于NP难问题，同样是一个难以在有限时间复杂度内完成的任务。不过我们可以通过启发式算法优化，在尽可能地保留信息的情况下，尝试将这个挖掘算法优化到我们可以接受、甚至能够接近线性级别的复杂程度，而这就是接下来我们这篇文章要讲的内容了。

这个时候，有认真读文章以及前言的同学应该看出来了，一个用来在线性数据结构上执行频繁序列匹配的PrefixSpan算法，怎么会被我拿来放到EFG上运行呢？PrefixSpan算法，又是怎么在一个有向图上跑起来的呢？

这里就不得不提到PrefixSpan算法的一个概念——"投影"。

PrefixSpan算法，本质上其实就是一个dfs（当然，PrefixSpan实际上不止可以用dfs实现，还有很多种方法可以实现这个算法，但考虑到我们不是教科书，这里就不细讲了），而dfs过程中每一步的状态，就是当前深度下的投影数据库，而dfs向下递归的步骤，就是扩展投影前缀的操作。而让PrefixSpan算法在有向图上运行的精髓，就在于在有向图变种算法中，扩展投影前缀的操作不再是向后移动一位序列指针，而是从当前节点出发，遍历所有出边指向的后继节点（实际算法实现中还应该有防环处理，这里省略掉这个步骤）。

此外，PrefixSpan算法还具备一个哈希等算法不具备的优点——那就是它天然容易实现节点跳过机制。

我们都知道，在实际运行环境中，一个程序的EFG可以是非常复杂的，而且对于狡猾的攻击者来说，他们执行恶意操作的时候，很有可能不是一气呵成，而是在恶意执行链中插入了很多的伪装API，来增加我们分析的复杂度。所以，让我们最后产出的挖掘算法具备节点跳跃的机制，是非常有必要的。

那么，PrefixSpan算法是如何自然地实现这个节点跳过机制的呢？核心还是"投影"这个概念。

说到这里，想必学习过PrefixSpan算法的同学已经能够想到应该怎么做了。还是对"扩展投影前缀"的操作下手，我们现在扩展投影前缀，不再是"从当前节点出发，遍历所有出边指向的后继节点"，而是以当前节点作为起点，执行有限深度的bfs（你要愿意也可以用dfs，但是嘛……嵌套递归，法力无边，不是我等贱民能够驾驭的）算法，把所有在指定步数范围内可达的后继节点都纳入候选集合，这样，扩展出的投影数据库中就天然拥有

了当前节点的下个、下下个、甚至是下下下个节点的数据，自然就可以进行带跳过的挖掘。

到这里，我们就已经讲清楚，一个PrefixSpan算法，是为何会被我拿到反病毒领域进行应用、如何能在一个有向图上运行的了。如果读者还有不太清楚的地方，可以查阅[引擎源码](https://github.com/ternaryop8479/csafestarligh/blob/main/src/tspm/trainer.cpp) (https://github.com/ternaryop8479/csafestarligh/blob/main/src/tspm/trainer.cpp)，引擎的代码里都有详细的注释，或许可以帮助理解（给个star叭求求了QAQ）。

不过到此为止，我们所讲解的都是如何让PrefixSpan在EFG上运行起来，但"理论可行"不等于"实践可行"，横在我们面前的还有最后一道门槛——那就是PrefixSpan算法的时间复杂度。一个标准的基于有向图运行的PrefixSpan变种算法，容易证明其最坏时间复杂度为 $O(\sum |V| \cdot d^L)$ （其中 $|V|$ 代表图节点数， $\sum |V|$ 为所有图的节点数之和， d 代表图的平均出度， L 代表最大链条深度）， L 当较大（一般设计为3~5，也就是时间复杂度为3次方~5次方）时，尽管 d 与 L 在实际数据集环境下都可以作为常数进行处理，但在EFG中，图的出度最大可以达到十几甚至数十，会导致 d^L 这个常数项在数据集扩张、图的最大出度不断更新的时候大到不可忍受的程度。尽管我们可以通过最小支持度等剪枝手段优化时间，但是整体的执行时间仍然是不可接受的。

那么，就没有一种合适的启发式算法，能够让我们在大规模EFG数据集上，高效地执行频繁API调用链挖掘吗？

当然有！不然我写这篇博客干嘛。不但有，而且非常简单，只需要引入一个参数——"最大扩张率"（Maximum Expansion Ratio），记作 R_e ，其合理最大值为1.0。而这个参数的定义也非常简洁直观：

$$R_e \geq \frac{|C|}{|P|}$$

其中 C 代表本次递归调用中，从当前前缀扩展出的合法候选API集合， P 代表当前前缀投影集。

当我们引入这个参数之后，就可以很容易证明，基于该参数的变种PrefixSpan算法（我将其命名为tosSpan!）可以达到近线性级别的复杂度，启发式证明如下。

首先，明确代码的执行结构，也就是简化地写出tosSpan算法的伪代码。定义 $\text{dfs}(P, \text{depth})$ 对前缀投影集 P 执行递归搜索，其中 S 代表最大跳过数， R_e 即最大扩张率 R_e ， C_i 即第 i 个候选 API 所持的投影集 C_i ：

```

1  dfs(P, depth):
2      对 P 中每个投影做受限 BFS (深度 ≤ S) :
3          收集后继API到候选投影集C
4      执行其他参数的剪枝筛选
5      若 |C| (|C|代表候选API种类数) > |P|·R_e: return // 膨胀率剪枝
6      遍历候选投影集C的候选API (每个API持有一个投影集, 记为C_i) :
7          push 前缀, 记录调用链
8          若 depth < L: dfs(C_i, depth+1)
9      pop 前缀
10
11  遍历EFG集合中所有EFG的所有节点:
12      将当前API加入到1-gram候选投影集C'
13
14  遍历1-gram候选投影集 (每个API持有一个投影集, 记为P'=C'_i) :
15      执行其他参数的剪枝筛选
16      dfs(P', 0)

```

我们首先对 $\text{dfs}(P, \text{depth})$ 的时间复杂度进行分析。

单投影bfs复杂度&单次dfs内部复杂度

设 V =单个EFG节点集, E =单个EFG边集。

对前缀投影集 P 中的任意一个投影 P_k , 其必定属于某一张EFG。对该投影 P_k 执行一次EFG上的bfs:

- ◇ bfs深度限制为 S , 每层深度遍历其子节点, 平均出度 $d = \frac{|E|}{|V|}$
- ◇ 对于一张EFG, bfs过程中访问的节点总数 $\leq |V|$

则对于单个投影, 其bfs复杂度:

$$T_{\text{proj}} \leq O(|V|)$$

单次dfs调用的内部复杂度:

$$\begin{aligned}
 T_{\text{layer}}(P) &= |P| \cdot T_{\text{proj}} \\
 &\leq |P| \cdot O(|V|) \\
 &= O(|P| \cdot |V|)
 \end{aligned}$$

单棵dfs树时间复杂度

设前缀投影集 P , 前缀 P 中每个API持有一个投影集 P_i

设根前缀投影集 P' ，则 $|P'|$ =根投影集大小。

这里需要引入一个关键的启发性假设，在极端情况下该假设不成立，但在朴素情况下是成立且与数据集大小无关的，且该假设的偏差常数（若有）不随样本数 $\sum |V|$ 增长（这也是该剪枝参数在大型数据集下仍然有效的核心）——对任意前缀投影集 P ，其展开出的所有候选投影集的节点总数与 P 的大小之比不超过 R_e ，即

$$\sum_{j=0} |C_j| \leq |P| \cdot R_e$$

（其中 $\sum_{j=0} |C_j|$ 代表当前层的某一个递归节点扩展出的所有候选API的总投影数）

根据该假设，注意到如下结论：

$$\sum \sum_{j=0} |C_j| \leq \sum |P| \cdot R_e$$

（其中 $\sum |P|$ 代表当前层的所有前缀投影集的大小之和，也就是当前层的前缀投影总数）

已知上一层的所有候选API的总投影数和当前层的前缀投影总数相等，故从根节点开始，递推可以得到：

$$\sum \sum_{j=0} |C_j| \leq |P'| \cdot R_e^i$$

则dfs树的总投影数（即所有节点的前缀投影之和,其中 P_t 代表dfs树的所有节点的前缀投影构成的总集合）：

$$|P_t| \leq \sum_{i=0}^L (|P'| \cdot R_e^i)$$

则单棵dfs树的时间复杂度：

$$\begin{aligned} T_{\text{tree}} &= T_{\text{proj}} \cdot |P_t| \\ &\leq T_{\text{proj}} \cdot \sum_{i=0}^L (|P'| \cdot R_e^i) \\ &\leq O(|V|) \cdot |P'| \cdot \sum_{i=0}^L R_e^i \\ &= O(|P'| \cdot |V| \cdot \sum_{i=0}^L R_e^i) \end{aligned}$$

总时间复杂度

设 $\sum |V|$ =EFG集的总节点数

注意到，对于所有根前缀投影集大小之和，其满足下式：

$$\sum |P'| = \sum |V|$$

于是得到所有dfs树的总时间复杂度：

$$\begin{aligned} T &= \sum T_{\text{tree}} \\ &\leq \sum O(|P'| \cdot |V| \cdot \sum_{i=0}^L R_e^i) \\ &= O(|V| \cdot \sum |P'| \cdot \sum_{i=0}^L R_e^i) \\ &= O(|V| \cdot \sum |V| \cdot \sum_{i=0}^L R_e^i) \end{aligned}$$

现在，我们已经推导出核心假设成立情况下的算法整体时间复杂度，接下来对其代入常数进行化简。

几何级数收敛

$$T = O\left(|V| \cdot \sum |V| \cdot \sum_{i=0}^L R_e^i\right)$$

已知 L 固定，在合理阈值下， $R_e \in (0, 1]$ 且一般情况下 R_e 不趋近于0（ $R_e = 1$ 的情况下会因 L 固定而有界 $L + 1$ ，而当 $R_e < 1$ 时， L 并不必然需要固定且有限），所以几何级数收敛至常数：

$$\sum_{i=0}^L R_e^i \leq \sum_{i=0}^{\infty} R_e^i = \frac{1}{1 - R_e}$$

代入：

$$T = O\left(|V| \cdot \sum |V| \cdot \frac{1}{1 - R_e}\right)$$

V 是常数

$|V|$ = 单图节点数上界（BFS 常数因子）。它不随样本数 $\sum |V|$ 增长——数据规模扩大是样本数 N 变多，单图大小不变，故：

$$T = O\left(|V| \cdot \frac{1}{1 - R_e} \cdot \sum_{\text{常数 } c} |V|\right)$$

合并常数

$$T = O\left(c \cdot \sum |V|\right) = O(\sum |V|)$$

即线性复杂度（或近线性复杂度）。且常见数据集下， $|V|$ 一般在数十至数百范围内， $\frac{1}{1-R_e}$ 在合理阈值下一般为个位数常数，因此常数 c 在朴素情况下应该在数百至数千的范围内，属于实践中可接受的复杂度。并且由于关键假设中，该假设的偏差常数不随数据集大小增长而变化，所以其在大数据集下的表现会远优于朴素算法（朴素算法的时间复杂度前文已经给出，为 $O(\sum |V| \cdot d^L)$ ）。

但必须声明的是，该算法并不总是能达到常数较低的线性复杂度，这取决于关键假设是否成立。在极端数据导致关键假设不成立的情况下（不过也真的只有极端数据集会发生退化了，至少在我自己的数据集上没有遇到过退化情况，并且根据实验数据观察，随着数据集规模增长，时间复杂度会从下方趋近于一条一次函数拟合线，这玩意儿退化的概率可能和快排退化的概率差不多），算法的时间复杂度会退化到与朴素算法同级的 $O(\sum |V| \cdot d^L)$ 。

这个时候，想必有些同学就会问了：这么激进的剪枝，是怎么做到在EFG里不剪掉有效恶意链条的呢？

首先，需要声明一点，该启发式算法并不保证必然不会剪掉恶意链条，但是和它剪掉的噪声链条数量相比，其剪掉的恶意链条数量可以说是微乎其微的。原因也很简单——在朴素的恶意与良性EFG中，恶意调用链在EFG中通常不会出现过于复杂的分支结构，如 `CreateRemoteThread->VirtualAllocEx->LoadLibrary` 这样的链条，它们在EFG中几乎就是一条没有分支节点的执行链。且对于被剪掉的链条，因为其是以分支过多的缘由被剪掉的，而根据观察，在朴素EFG中，分支过多的链条和节点，通常都属于无恶意的通用调用，所以自然就不需要担心这个参数会咋咋剪掉有效的调用链。此外，由于被剪枝的链条大多属于无恶意的通用调用，这个参数在大部分情况下反而是可以对挖掘出的数据库实现提纯功能的。

另外还需要注意的一点是，因为纯恶意调用链肯定不会在良性程序中出现，但是良性调

用理论而言可以在恶意程序中出现，所以我们将纯恶意或可以被应用于恶意行为的调用链视为有效链条，而其他的链条都视为无效，即噪声链条。

至于测试数据嘛~我自己的七万数据集（恶意四万良性三万，详细数据可以看[代码仓库](https://github.com/ternaryop8479/csafestarligh/blob/main/ModelList.md)（<https://github.com/ternaryop8479/csafestarligh/blob/main/ModelList.md>）），测试平台为超算互联网的免费Intel Xeon 7285H 32C机，全核心训练，训练参数为[默认参数](https://github.com/ternaryop8479/csafestarligh/blob/main/default_train_param.conf)（https://github.com/ternaryop8479/csafestarligh/blob/main/default_train_param.conf），交叉验证三次+训练一次+LightGBM训练，总共49分49秒，内存峰值16G（包含EFG本体），如果只看最后的一次七万样本全量训练的话，黑样本数据集挖掘时长4308ms，白样本数据集挖掘时长2684ms，算上数据集初始化，总训练时长也只有11453ms，足以证明该算法有效。至于AUC嘛~大家看[最新模型数据](https://github.com/ternaryop8479/csafestarligh/blob/main/ModelList.md)（<https://github.com/ternaryop8479/csafestarligh/blob/main/ModelList.md>）就够了，毕竟在机器学习杀毒引擎领域，只喂PE特征给LightGBM是不可能做到比较好的水平的。

算法设计

奇思妙想

反病毒

频繁序列挖掘

CSafe-Starlight

CFTQ云

C++

toの随记

本文采用知识共享署名协议，转载请注明出处。

链接：</archives/BcOfhQzF>